

A Test Suite Reduction Approach Based on Pairwise Interaction of Requirements

Xiang Chen[†] Lijiu Zhang[†] Qing Gu^{†*} Haigang Zhao[‡] Ziyuan Wang[‡] Xiaobing Sun[‡] Daoxu Chen[‡]

[†]State Key Laboratory for Novel Software Technology
Nanjing University, Nanjing, China
chenxiang@dislab.nju.edu.cn

[‡]School of Computer Science and Engineering
Southeast University, Nanjing, China
sundomore@seu.edu.cn

ABSTRACT

Test suite reduction is one of the effective techniques to reduce the cost of regression testing. In particular, it tries to identify and remove redundant test cases according to a specific test coverage criterion. However, the excessive reduction in test cases may also significantly weaken the fault detection ability of the original test suite. In this paper, we conjecture that covering interaction of test requirements can improve the fault detection ability and propose a new test suite reduction approach. As a preliminary study, we firstly propose a pairwise interaction based coverage criterion (PWIC). Then we propose a pairwise interaction of requirements based test suite reduction approach (PWIR). To assess the feasibility and usefulness of our proposed approach, we implement PWIR approach and conduct an empirical study on seven real C programs. After analyzing the results of the empirical studies, we conclude that our approach can improve the fault detection ability without severely increasing the reduced test suite size.

Categories and Subject Descriptors

D.2.5 [Software Engineering]: Testing and Debugging

General Terms

Regression Testing

Keywords

Test Suite Reduction, Pairwise Interaction of Requirements

1. INTRODUCTION

During software maintenance phase, software under test (SUT) often undergoes a series of evolutions. These evolutions may be caused by new faults or new features. Regression testing is used to retest evolved SUT and guarantee

the correctness of both new features and unchanged parts of SUT. However, during the software evolutionary process, test suite size will continually increase as new test cases are added. If the cost of running a single test case is high or the test cases are manually maintained (e.g., tests oracle construction), the cost of regression testing will be prohibitive. Test suite reduction (also known as test suite minimization) is one of the important regression testing techniques. This technique aims to identify and eliminate redundant test cases according to a specific test coverage criterion (also known as test adequacy criterion) and thus can save the cost of regression testing.

In previous research work, some researchers focused on proposing novel approaches to reduce test suites according to a single coverage criterion [2][6][16][19]. However, in earlier research work, Rothermel et al. conducted a set of empirical studies and found that test suite reduction can provide significant savings in size but also significantly lose fault detection ability (FDA) of original test suites [17]. In recent research work, Namin and Andrews observed that both test suite size and requirement coverage are important to test suite FDA [15]. These empirical studies show that to improve test suites FDA, it is necessary to propose systematic strategies to augment reduced test suites by adding some test cases. Up to now, some strategies have been proposed. Jeffrey and Gupta proposed the concept of selective redundancy based on multiple test coverage criteria [11]. Black et al. performed test suite reduction over a bi-criteria binary integer linear program model [1]. Hsu and Orso extended the work [1] and developed a MINTS tool to support test suite reduction with the ability of encoding a wide spectrum of optimization objectives [10].

Different from previous research work, in this paper, we propose another lightweight test suite reduction approach. Traditional approaches only consider the coverage of individual requirements. The key intuition of our approach is based on the conjecture that some faults have relationship with interaction of requirements and these interactions should be considered in the test suite reduction. Though in real testing scenarios, the interaction strength of different requirements may vary (such as pairwise interaction, 3-way interaction). But as a preliminary investigation, here we only consider pairwise interactions. In this paper, we firstly propose a new pairwise interaction of requirements based coverage criterion (PWIC) and then propose a pairwise interaction based test suite reduction approach (PWIR). In particular, given a test suite T and a set of test requirements R generated by a given test coverage criteria C , we can gather the coverage infor-

*Correspondence author. E-mail: guq@nju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

SAC'11 March 21-25, 2011, TaiChung, Taiwan.

Copyright 2011 ACM 978-1-4503-0113-8/11/03 ...\$10.00.

mation of all the test cases in T . In our approach, we first use a modified HGS (HGS is a classical test suite reduction algorithm [6] and some modifications will be illustrated in Section 3.2) to generate a minimized test suite T_{min} . However, T_{min} often cannot cover all the interactions generated by PWIC, then we augment T_{min} by choosing some remaining test cases in T to cover all the remaining interactions and return this augmented test suite T_{aug} . To the best of our knowledge, our paper is the first to perform test suite reduction considering the interaction of requirements.

To verify the feasibility and usefulness of our approach, we implement PWIR approach and perform an empirical study on seven real C programs known as Siemens suite. Our empirical study is designed to make a comparison with HGS and other two random approaches Random and HGSRAdd. Final results conclude that our approach can improve FDA without severely increasing the size of reduced test suites.

The main contributions of this paper include:

- We propose a new test coverage criteria PWIC concerning pairwise interaction of requirements and a new test reduction approach PWIR based on PWIC.
- We conduct an empirical study to verify the effectiveness of our approach. To show the advantage of our approach, we propose other two random approaches Random and HGSRAdd as a baseline. Final results show that PWIR can improve FDA without severely increase the size of reduced test suites.

The rest of this paper is organized as follows. Section 2 provides the background of test suite reduction and gives a motivation example to introduce the intuition of our approach. Section 3 presents the detail of our reduction approach PWIR. Section 4 describes our empirical study including experiment subjects, experiment design, data analysis, and threats to validity. Section 5 summarizes the related work of test suite reduction. Finally, section 6 gives the conclusion and future work.

2. BACKGROUND

In this section, we firstly introduce the preliminary knowledge of test suite reduction and then use a simple example to show the motivation of our approach.

2.1 Test Suite Reduction

Regression testing is frequently performed as software continually evolves. However, retesting all the previous test cases is often infeasible. To tackle this issue, recent research work focuses on test cases reduction, selection, augmentation, and prioritization [7]. In this paper, we focus on test suite reduction. This problem can be formally defined as follows:

Given: A test suite $T = \{t_1, t_2, \dots, t_m\}$, a set of test requirements $R = \{r_1, r_2, \dots, r_n\}$ generated by a test coverage criterion C , and subsets $\{T_1, T_2, \dots, T_n\}$ of T , one associated with each of the r_i s such that any one test $t_j (t_j \in T_i)$ covers requirement r_i .

Problem: To find a subset of T with the minimum cardinality that covers all the requirements covered by T .

This definition is firstly proposed by Haralad et al. [6] and then widely accepted by other researchers. Test suite reduction can identify and remove redundant test cases. Therefore, it can greatly save the cost of software testing. However, by reducing this optimization problem to the minimal

set cover problem in polynomial time, this problem can be demonstrated to be NP-Complete [21]. Up to now, several approaches have been proposed to solve this problem [1][2][6][10][11][16][17][19] and a brief summarization can be found in Section 5.

2.2 Motivating Example

In this subsection, we use a simple motivating example shown in Table 1 to illustrate the limitation of existing work and introduce our approach.

Table 1: A Motivating Example

	r_1	r_2	r_3	r_4	r_5	f_1	f_2	f_3
t_1	X		X			X		
t_2		X		X	X		X	
t_3		X	X					X
t_4		X		X				
t_5		X			X	X		

In this example, we assume that this program has one version P without faults and three faulty versions $V = \{v_1, v_2, v_3\}$. Here the faulty version v_i has only one fault f_i . The test suite T has 5 test cases and the program P has 5 test requirements to be covered. Table 1 records requirement coverage information after executing these test cases on P and fault detection information on faulty versions V . For example, in Table 1, the test case t_1 covers the test requirements $\{r_1, r_3\}$ of the program P and it can detect the fault f_1 in the faulty version v_1 .

Since we cannot know the fault detection information of test cases on faulty versions in advance, here we perform test suite reduction according to the coverage information on the program P . In this example, if we use HGS algorithm [6], we will get the reduced test suite $T_{min} = \{t_1, t_2\}$. The percentage of test suite size reduction is $\frac{5-2}{5} \times 100 = 60$. However, T_{min} cannot detect the fault f_3 in the faulty version v_3 , therefore the percentage of FDA reduction is $\frac{3-2}{3} \times 100 = 33.33$. Meanwhile we find that although T_{min} covers all the individual requirements, it cannot cover all the pairwise interactions of requirements covered by T . To cover the remaining interaction $\langle r_2, r_3 \rangle$, we need to augment T_{min} by adding the test case t_3 and get this augmented test suite T_{aug} . As a result, T_{aug} can detect all the faults by adding only one test case. This example is deliberately constructed to illustrate the intuition of our approach and in the following empirical study, we will use real complex C programs to verify the effectiveness of our approach.

3. PWIR APPROACH

In this section, we first show the framework of our PWIR approach and then introduce the implementation in detail.

3.1 The Framework of PWIR Approach

When test suite reduction only concerns one test coverage criterion, it can remove numerous redundant test cases but at the price of sacrificing the whole test suite FDA [17]. To tackle this issue, we need to consider other new coverage criteria. In this paper, the new coverage criterion is designed with respect to the observation that covering all the interactions of requirements helps to detect more faults than only covering all the individual requirements. As a preliminary

study, we only consider pairwise interactions in this research. We define this new coverage criterion as follows:

Definition 1: (Pairwise Interaction Based Coverage Criterion, PWIC): PWIC generates all the interactions of arbitrary two requirements (i.e., pairwise interaction) covered by test suite T .

Given the test requirements R ($|R| = n$), the number of all possible pairwise interactions is $n * (n - 1) / 2$. However, we only need to consider all the pairwise interactions covered by T and use $I_{feasible}$ to denote this set. In our motivating example, $I_{feasible} = \{ \langle r_1, r_3 \rangle, \langle r_2, r_3 \rangle, \langle r_2, r_4 \rangle, \langle r_2, r_5 \rangle, \langle r_4, r_5 \rangle \}$.

Based on the above coverage criterion, we propose our PWIR approach. The pseudocode of PWIR can be found in Algorithm 1. First, given a program P and a specific test coverage criterion C , a set of test requirements $R_{uncovered}$ and feasible pairwise interactions $I_{feasible}$ are generated. Then, the algorithm is divided into two steps. In step 1, it aims to cover all the requirements in $R_{uncovered}$ (Lines 5-12). According to the coverage information of test cases in test suite T , it applies the function *chooseNextTestMin*, which will be further illustrated in subsection 3.2, to choose test cases. After selecting a test case *test*, it uses the function *compCoveredReqs* to compute the requirements covered by *test* and the function *compCoveredPairs* to compute the pairwise interactions of requirements covered by *test*. After step 1, it can get a minimized test suite T_{min} and the pairwise interaction set $I_{covered}$ covered by T_{min} . In step 2, it first computes uncovered pairwise interaction set $I_{uncovered}$ (i.e., $I_{uncovered} = I_{feasible} - I_{covered}$). If $I_{uncovered}$ is not empty, it augments test suite (Lines 14-20). It uses the function *chooseNextTestAug*, which will also be illustrated in subsection 3.2, to augment T_{min} by adding selected test cases from $(T - T_{min})$. After covering all the interactions in $I_{uncovered}$, the test suite T_{aug} is returned as the output.

3.2 The Implementation of PWIR Approach

In this subsection, we introduce the implementation details of our approach PWIR and mainly focus on the test suite minimization algorithm and the test suite augmentation algorithm.

First we introduce the test suite minimization algorithm in step 1. Given test requirement coverage information of test cases in T , this algorithm aims to generate a test suite T_{min} that covers all the feasible requirements R . To choose a test case for T_{min} , we use algorithm 2 to implement the function *chooseNextTestMin* in algorithm 1. The algorithm is adapted from HGS but takes pairwise interaction coverage into account in the case of ties. The input of this algorithm is the remaining unchosen test cases T_{remain} , uncovered requirements $R_{uncovered}$, and uncovered interactions $I_{uncovered}$. The algorithm outputs the chosen test case t and puts t to T_{min} . First, we use the function *computeCard* to compute the cardinality of each requirement in $R_{uncovered}$ (i.e., the number of test cases in T_{remain} covering this requirement) (Lines 1-3). Then we select the test cases T_1 from T_{remain} covering the most requirements with the minimum cardinality (Lines 4-5). If T_1 contains only one test case, the algorithm will return this test case (Line 14). Otherwise in the case of ties, the algorithm chooses the test cases T_2 from T_1 that cover the most pairwise interactions in $I_{uncovered}$ (Line 7). If ties still exist, to break the ties, the algorithm will first choose the test cases T_3 from T_2 that cover the most

Algorithm 1 PWIR Algorithm

Input: A program P , a test suite T , a given coverage criterion C .
Output: A reduced test suite T_{aug} .

```

1:  $T_{min} \leftarrow \emptyset$ ;
2: Generate test requirements  $R_{uncovered}$  according to  $P$  and  $C$ ;
3: Generate feasible pairwise interactions  $I_{feasible}$ ;
    $I_{uncovered} \leftarrow I_{feasible}$ ;
4: // step 1: test suite minimization algorithm;
5: while  $R_{uncovered} \neq \emptyset$  do
6:    $test \leftarrow \text{chooseNextTestMin}(T - T_{min}, R_{uncovered}, I_{uncovered})$ ;
7:    $R_{test} \leftarrow \text{compCoveredReqs}(test)$ ;
8:    $I_{test} \leftarrow \text{compCoveredPairs}(test)$ ;
9:    $R_{uncovered} \leftarrow R_{uncovered} - R_{test}$ ;
10:   $I_{uncovered} \leftarrow I_{uncovered} - I_{test}$ ;
11:   $T_{min} \leftarrow T_{min} \cup \{test\}$ ;
12: end while
13: // step 2: test suite augmentation algorithm;
14:  $T_{aug} \leftarrow T_{min}$ ;
15: while  $I_{uncovered} \neq \emptyset$  do
16:    $test \leftarrow \text{chooseNextTestAug}(T - T_{aug}, I_{uncovered})$ ;
17:    $I_{test} \leftarrow \text{compCoveredPairs}(test)$ ;
18:    $I_{uncovered} \leftarrow I_{uncovered} - I_{test}$ ;
19:    $T_{aug} \leftarrow T_{aug} \cup \{test\}$ ;
20: end while
21: return  $T_{aug}$ ;

```

requirements with the next higher requirement cardinality. If the number of T_3 is still greater than 1, we choose the test cases from T_3 that cover the most pairwise interactions in $I_{uncovered}$. Breaking ties repeats until the cardinality equals the maximum cardinality and test cases cover the same pairwise interactions in $I_{uncovered}$. In this case, the algorithm randomly selects a test case from the tied test cases (Line 9).

Algorithm 2 *chooseNextTestMin* function

Input: Remaining test cases T_{remain} , uncovered requirements $R_{uncovered}$, uncovered pairwise interactions $I_{uncovered}$.
Output: A test case t .

```

1: for each  $r_i \in R_{uncovered}$  do
2:    $card_{r_i} \leftarrow \text{computeCard}(r_i)$ ;
3: end for
4:  $minCard \leftarrow \min\{card_{r_i} | r_i \in R_{uncovered}\}$ ;
5: Select test cases  $T_1 (T_1 \subseteq T_{remain})$  that cover the most requirements in  $\{r_i | card_{r_i} = minCard \wedge r_i \in R_{uncovered}\}$ ;
6: if  $|T_1| \neq 1$  then
7:   Select test cases  $T_2 (T_2 \subseteq T_1)$  that cover the most pairwise interactions in  $I_{uncovered}$ ;
8:   if  $|T_2| \neq 1$  then
9:     Use test cases tie-break and return test case  $t$ ;
10:  else
11:    return test case  $t \in T_2$ ;
12:  end if
13: else
14:  return test case  $t \in T_1$ ;
15: end if

```

Second, we introduce the test suite augmentation algorithm in step 2. This algorithm aims to augment test suite T_{min} by adding selected test cases to cover all the remaining pairwise interactions in $I_{uncovered}$. To choose a test case to T_{aug} , we use algorithm 3 to implement the function *chooseNextTestAug* in algorithm 1. The algorithm is also motivated by HGS and before running this algorithm, we need to gather the interaction coverage information of each test case in $(T - T_{min})$. The input of this algorithm is the remaining unchosen test cases T_{remain} and the remaining pairwise interactions $I_{uncovered}$. The algorithm outputs a test case t and adds it to T_{aug} . First, we use the function *computeCard* to compute the cardinality of each interaction in $I_{uncovered}$ (i.e., the number of test cases in T_{remain} covering this interaction) (Lines 1-3). Then we select the test cases T_1 of T_{remain} covering the most interactions with the minimum cardinality. In the case of ties, the algorithm chooses the test cases that occur most frequently at the next higher cardinality. Breaking ties repeats until the cardinality reaches the maximum cardinality and the algorithm randomly selects a test case from the tied test cases (Lines 4-10).

Algorithm 3 *chooseNextTestAug* function

Input: Remaining test cases T_{remain} , uncovered pairwise interactions $I_{uncovered}$.

Output: A test case t .

```

1: for each  $p_i \in I_{uncovered}$  do
2:    $card_{p_i} \leftarrow computeCard(p_i)$ ;
3: end for
4:  $minCard \leftarrow \min\{card_{p_i} | p_i \in I_{uncovered}\}$ ;
5: Select test cases  $T_1 (T_1 \subseteq T_{remain})$  that cover the most
   pairwise interactions in  $\{p_i | card_{p_i} = minCard \wedge p_i \in I_{uncovered}\}$ ;
6: if  $|T_1| \neq 1$  then
7:   Use test cases tie-break and return test case  $t$ ;
8: else
9:   return test case  $t \in T_1$ ;
10: end if

```

Finally, we analyze the worst-case time complexity of PWIR approach. Let n denote the number of test requirements in R , m denote the number of test cases in T , and $maxCard$ denote the maximum cardinality of test requirements. Based on the analysis result of HGS algorithm [6], the worst-case time complexity of Algorithm 2 is $O((n+m \times n^2) \times maxCard)$, because this algorithm takes the pairwise interaction coverage into account in test cases tie-break and the number of interactions is $n \times (n-1)/2$ at most. The complexity of Algorithm 3 is $O((n^2 + m) \times maxCard)$, because this algorithm chooses the next test case according to the cardinality of pairwise interactions. The worst-case time complexity of PWIR approach is $O((n^4 + m \times n^3) \times maxCard)$. But in practice, our approach consumes less time, one reason is that the size of feasible pairwise interaction set $I_{feasible}$ is far less than $n \times (n-1)/2$.

4. EMPIRICAL STUDIES

To evaluate the effectiveness of PWIR approach, we design and perform a set of empirical studies to answer the following two research questions:

RQ1: To what extent does the size of reduced test suites

generated by PWIR approach increase compared to HGS approach?

RQ2: Can PWIR approach improve the fault detection ability without severely increasing the size of reduced test suites?

In this section, we first introduce the experiment subjects. Then we show the experiment design, gather and analyze the data generated from the experiments to answer the preceding two research questions. Finally we analyze main threats to the validity of our empirical studies.

4.1 Experiment Subjects

Subjects in our empirical studies are called Siemens suite. Table 2 provides summary information about these seven subjects. The information includes the name, the number of lines of code (LOC), the number of faulty versions, the size of test pool, and the brief description of each subject. LOC of these seven subjects ranges from 141 to 512. The size of test pool is large, at most 5542 and at least 1052. Each subject has multiple faulty versions between 7 and 41. Each faulty version has only one fault. These subjects are commonly used to determine the effectiveness of proposed test suite reduction approaches [1][10][11][13][15][17][19]. We downloaded these subjects from Subject Infrastructure Repository (SIR) at University of Nebraska-Lincoln¹ [4].

Table 2: Experiment Subjects

Subject	LOC	# Tests	# Ver	Description
printtok	472	4130	7	Lexical Analyzer
printtok2	399	4115	10	Lexical Analyzer
replace	512	5542	32	Pattern Replacement
schedule	292	2650	9	Priority Scheduler
schedule2	301	2710	10	Priority Scheduler
tcas	141	1608	41	Altitude Separation
totinfo	440	1052	23	Information Measure

4.2 Experiment Design

Since there exists randomness variation inherent in our approach, we performed our experiments 1000 times independently to get a high statistical confidence. In each independent experiment, we randomly generate a test suite from the test pool and apply our PWIR approach to this test suite. This test suite generation process is briefly summarized as follows: First we determine the test suite size n . This size is randomly determined in the interval of 0 and 0.5 times LOC of the subject. Then the test suite is constructed by randomly selecting n test cases from the test pool. Finally, if the generated test suite is not adequate according to the given coverage criterion, we will augment this test suite by adding additional test cases to guarantee the 100% coverage of feasible test requirements. This experiment design is motivated by Rothermel et al [17]. We use box plox² to depict the distribution of our generated 1000 test suites size

¹<http://sir.unl.edu/php/index.php>

²A boxplot is a convenient way to graphically depicting groups of numerical data in descriptive statistics. It consists of five most important sample percentiles: the sample minimum, the lower quartile, the median, the upper quartile and the sample maximum. The box's height spans the central 50% of the data. Its upper and lower ends mark the upper quartile and the lower quartile. The middle of the three horizontal lines within the box represents the median.

in Figure 1. We also use box plot to show the distribution of percentage in FDA of these test suites compared to the original test pool in Figure 2.

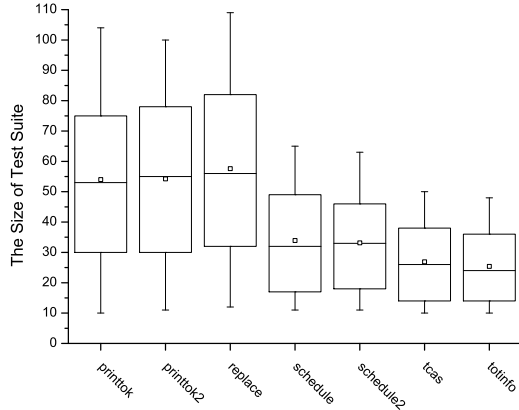


Figure 1: The distribution of generated 1000 test suites in size for each of seven subjects

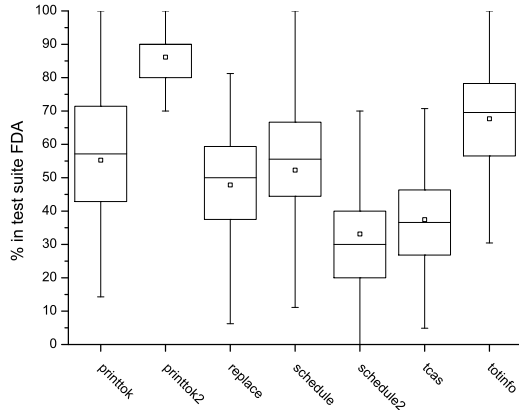


Figure 2: The distribution of percentage in FDA of generated 1000 test suites compared to the original test pool for each of seven subjects

To measure the effectiveness of our approach, we need a set of metrics to evaluate the empirical results. In particular, we need to measure the cost and FDA of our approach. Test suite reduction can save the time cost of regression testing. To test real-world softwares, the time cost of test suites includes test suite setup time, test cases execution time, and test result examination time. In this paper, we only consider test suite size and adopt the percentage of the reduced test suite $T_{reduced}$ size relative to the original test suite T as a metric to measure the time cost. The formula is $\frac{|T_{reduced}|}{|T|} \times 100$. According to the experiment design, we know that each of faulty versions only has one fault. To measure the percentage of the reduced test suite $T_{reduced}$ FDA

compared to the original test suite T , we adopt the metric defined by the formula $\frac{|F_{reduced}|}{|F|} \times 100$. In this formula, $|F|$ denotes the number of faulty versions which can be detected by the test suite T over all the faulty versions. $|F_{reduced}|$ denotes the number of faulty versions which can be detected by the reduced test suite $T_{reduced}$. In this metric, we assume that each fault has the same severity.

4.3 Results and Analysis

In this subsection, we gather and analyze the data of empirical studies to answer RQ1 and RQ2.

4.3.1 Test Suite Size Analysis

To answer RQ1, we implement HGS algorithm [6] and treat it as a baseline. Here for each of seven subjects, we use HGS and PWIR approaches to reduce 1000 test suites respectively. In this paper, the used test coverage criterion is statement coverage. After analyzing the data, the average reduced test suite size and percentage are shown in Table 3 and the distribution of the percentage of these reduced test suites size compared to the original test suites is shown in Figure 3.

Table 3: Experiment Results for the Test Suite Size

Subject	$ T $	$ T_{reduced} $		% in size	
		HGS	PWIR	HGS	PWIR
printtok	53.99	7.67	14.93	21.01	36.31
printtok2	54.23	6.16	14.43	17.30	35.04
replace	57.59	11.07	22.74	26.19	47.25
schedule	33.92	4.29	5.41	17.35	20.98
schedule2	33.15	5.08	6.41	20.25	24.59
tcas	26.90	4.00	5.14	18.94	24.04
totinfo	25.40	5.10	9.32	25.62	42.76

From table 3, the average value of reduced test suite size generated by HGS is less than the size generated by PWIR. This result meets our expectation. Because in addition to cover all the individual requirements R , PWIR still needs to cover all the feasible pairwise interactions of requirements in $I_{feasible}$. The coverage of these additional requirements needs more test cases. In some subjects (e.g., schedule, schedule2, tcas, and totinfo), the increased size is relatively small, only from 1 to 4. In other subjects, the increased size is relatively large but our PWIR approach still achieves high percent of test suite size reduction which ranges from 52.75 to 79.02.

4.3.2 Fault Detection Ability Analysis

To answer RQ2, we introduce other two approaches for comparison to show the advantage of our PWIR approach. Given a test suite T , the first approach firstly sets $T_{reduced}$ to be empty, then chooses test cases from $(T - T_{reduced})$ randomly and adds them to $T_{reduced}$ until the size of $T_{reduced}$ is the same as the size of reduced test suite generated by PWIR. We refer to this approach as Random Approach. The second approach first generates a reduced test suite $T_{reduced}$ by using HGS algorithm and then augments $T_{reduced}$ by choosing test cases from $(T - T_{reduced})$ randomly until the size is the same as the test suite size generated by PWIR. We refer to this approach as HGSRAAdd Approach.

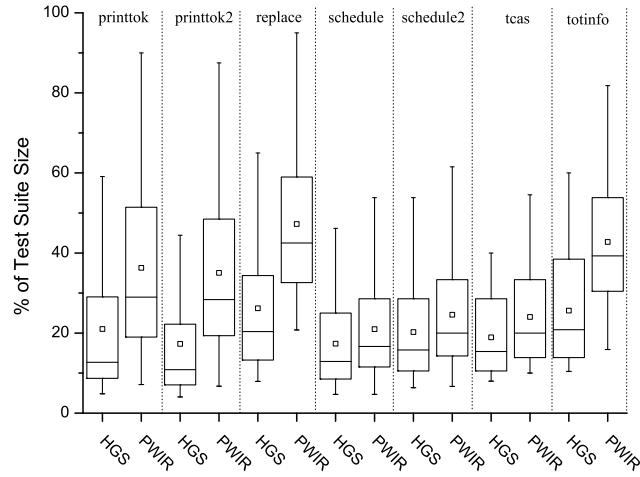
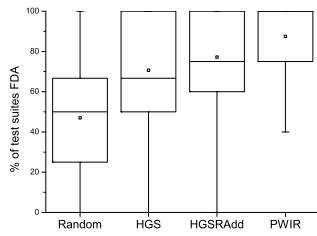
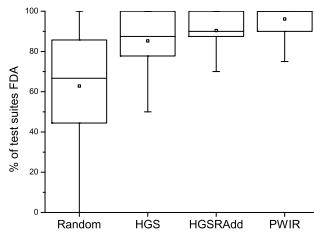


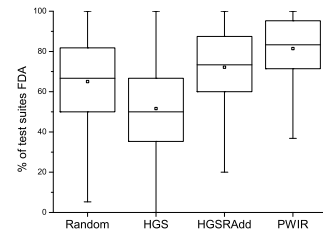
Figure 3: The Distribution of Percentage in Reduced Test Suites Size for Each of Seven Subjects



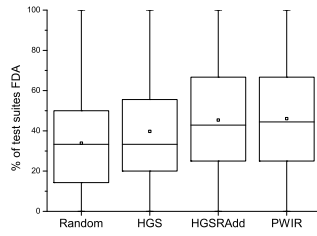
(a) printtok



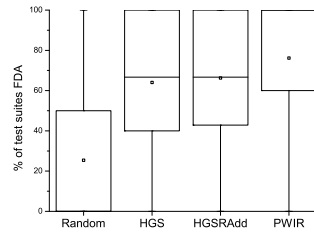
(b) printtok2



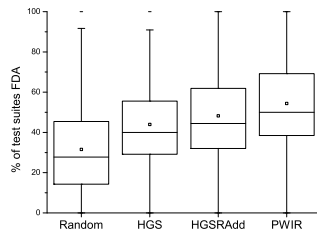
(c) replace



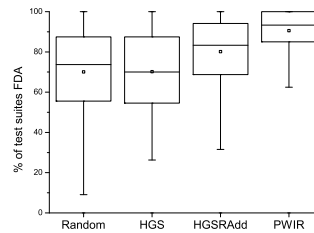
(d) schedule



(e) schedule2



(f) tcas



(g) totinfo

Figure 4: The Distribution of Percentage in Reduced Test Suites FDA for Each of Seven Subjects of Random, HGS, HGSRAAdd, and PWIR

Table 4: Experiment Results for the Test Suite FDA of Random, HGS, HGSRAAdd, and PWIR

Subject	F	$F_{reduced}$				% in FDA			
		Random	HGS	HGSRAAdd	PWIR	Random	HGS	HGSRAAdd	PWIR
printtok	3.87	1.70	2.60	2.88	3.32	47.00	70.64	77.23	87.48
printtok2	8.61	5.39	7.32	7.78	8.30	62.82	85.23	90.42	96.11
replace	15.31	9.65	7.42	10.74	12.24	65.02	51.61	72.16	81.46
schedule	4.71	1.54	1.86	2.13	2.15	33.92	39.73	45.40	46.08
schedule2	3.31	0.85	2.10	2.18	2.52	25.42	64.05	66.26	76.16
tcas	15.35	4.50	6.14	6.79	7.68	31.53	43.97	48.27	54.38
totinfo	15.56	10.86	10.71	12.35	14.10	70.07	70.20	80.14	90.55

The average value of reduced test suite FDA and percentage are shown in Table 4. The distribution of these data is shown in Figure 4. From these statistical data, we can find that except subjects replace and totinfo, Random approach always has poorest FDA, even poorer than HGS (Notice that the reduced test suite size of Random always greater than the size of HGS). In the case of the same size of reduced test suites, our approach PWIR has the best FDA among all these seven subjects. HGS approach has the poorest FDA and HGSRAAdd is between PWIR and HGS. These empirical results conclude that in the procedure of test suite reduction, considering pairwise interaction of requirements coverage can effectively improve the FDA of reduced test suites.

4.4 Threats to Validity

In this subsection, we discuss some potential threats to our empirical studies.

Threats to internal validity mainly come from the corrupted data or incorrect program implementation. To avoid these threats, we prepared our data carefully and tested our programs with simple programs.

Threats to external validity affect the results generalization. The primary threat for our study is the representativeness of our subjects. However, these subjects are commonly used by other researchers. Meanwhile, our conclusion may not be applicable to some subjects written by other programming languages, such as C++, Java, or C#. Another threat is that we only adopt HGS as our test suite reduction approach. Using other reduction approaches may affect the conclusion of our paper.

Threats to conclusion validity arise when accounting for the random variation in our approach. To overcome these threats, we need more sophisticated statistical hypothesis tests to have a higher confidence in our results.

5. RELATED WORK

Nowadays, test suite reduction is still a hot research topic in regression testing. Research achievements have been successfully applied to different domains, such as web applications [18], GUI applications [14], and fault localization [22][23]. Previous research work can be briefly summarized into three lines.

The first line of research focuses on proposing effective approaches based on a single test coverage criterion. Horgan and London used linear programming to conduct test suite minimization based on data flow coverage [9]. Harrold et al. introduced a classical greedy approach called HGS [6]. Chen and Lau proposed two greedy algorithms called heuris-

tic GE and GRE [2]. Jones and Harrold developed two techniques with respect to the modified condition/decision coverage(MC/DC) criterion [12]. Tallam and Gupta proposed an approach based on formal concept analysis [19]. Because test suite reduction is a typical combinatorial optimization problem, meta-heuristic search techniques can be used to solve this problem, such as, simulated annealing and genetic algorithm [16].

The second line of research focuses on performing test suite reduction based on multiple coverage criteria. Jeffrey and Gupta extended HGS and proposed the concept of selective redundancy. They attempted to select additional test cases that are redundant to a particular coverage criterion but are not redundant with respect to one or more other coverage criteria [11]. Black et al. proposed a reduction approach using bi-criteria binary ILP model (i.e., branch coverage and def-use pair coverage) [1]. Hsu and Orso extended the work [1] and could solve a wide range of multi-criteria test suite reduction problems. Their tool MINTS could encode these problems as binary ILP problems and leveraged existing modern ILP solvers to find optimal minimal solutions [10].

The remaining work can be categorized into the third line of research. Vaysburg et al. used EFSM dependence analysis to conduct requirement-based test suite reduction [20]. Harder et al. generated, augmented, and minimized test suites by the operational difference technique [5]. Martina and Bertolino introduced the notion of spanning sets of entities and this notion can be used to perform test suite reduction [13]. Heimdahl and George conducted test suite reduction with respect to given specification-based criteria that are based on formal models of the software [8]. Chen et al. proposed an approach based on requirement relation contraction for specification-based testing [3].

However, all of the existing approaches do not consider the interaction of test requirements and our paper is the first attempt to perform test suite reduction considering the interaction of test requirements and gives the initial experiment results.

6. CONCLUSION

The fault detection ability of test suites may be severely compromised by test suite reduction [17]. Based on the conjecture that covering interaction of requirements can help to detect more faults, we propose a pairwise interaction based coverage criterion PWIC and a new test suite reduction approach PWIR. Final empirical studies verify the effectiveness of our approach.

As a preliminary research, there are some issues needed

to be solved in our future work. First, we want to investigate how to set reasonable interaction strength (such as pairwise interaction, 3-way interaction) for different requirement subsets and therewith propose new approaches. Second, we want to apply our approach to other subjects written by different programming languages in SIR and verify the effectiveness of our approach. Last but not the least, we want to apply our approach to larger, real-world applications.

Acknowledgment

We would like to thank Jingxian Qi, Xing Shang, and Zhanqi Cui for their comments on this work. We would also like to thank anonymous reviewer for their insightful and constructive comments to improve this paper. This work is supported in part by the NSFC Project under Grant No. 60873027, the National High-Tech Research and Development Plan of China (863) under Grant No. 2006AA01Z177, and the 973 Program of China under Grant No. 2009CB320705.

7. REFERENCES

- [1] J. Black, E. Melachrinoudis and D. Kaeli. Bi-criteria Models for All-uses Test Suite Reduction. In *Proceedings of the 26th International Conference on Software Engineering*, pp. 106-115, 2004.
- [2] T.Y. Chen and M.F. Lau. A New Heuristic for Test Suite Reduction. *Information and Software Technology*, 4(5-6): pp. 347-354, 1998.
- [3] Z.Y. Chen, B.W. Xu, X.F. Zhang and C.H. Nie. A Novel Approach for Test Suite Reduction Based on Requirement Relation Contraction. In *Proceedings of the 2008 ACM Symposium on Applied Computing*, pp. 390-394, 2008.
- [4] H. Do, S. Elbaum and G. Rothermel. Supporting Controlled Experimentation with Testing Techniques: An Infrastructure and its Potential Impact. *Empirical Software Engineering*, 10(4): pp. 405-435, 2005.
- [5] M. Harder, J. Mellen and M.D. Ernst. Improving Test Suites via Operational Abstraction. In *Proceedings of the 25th International Conference on Software Engineering*, pp. 60-71, 2003.
- [6] M.J. Harrold, R. Gupta and M.L. Soffa. A Methodology for Controlling the Size of a Test Suite. *ACM Transactions on Software Engineering and Methodology*, 2(3): pp. 270-285, 1993.
- [7] M.J. Harrold and A. Orso. Retesting Software During Development and Maintenance. In *Frontiers of Software Maintenance*, pp. 99-108, 2008.
- [8] M. Heimdahl and D. George. Test-Suite Reduction for Model Based Tests: Effects on Test Quality and Implications for Testing. In *Proceedings of 19th International Conference on Automated Software Engineering*, pp. 176-185, 2004.
- [9] J.R. Horgan and S. London. Data Flow Coverage and the C Language. In *Proceedings of the Symposium on Testing, Analysis and Verification*, pp. 87-97, 1991.
- [10] H. Hsu and A. Orso. MINTS: A General Framework and Tool for Supporting Test-suite Minimization. In *Proceedings of the 31st International Conference on Software Engineering*, pp. 419-429, 2009.
- [11] D. Jeffrey and N. Gupta. Improving Fault Detection Capability by Selectively Retaining Test Cases During Test Suite Reduction. *IEEE Transactions on Software Engineering*, 33(2), pp. 108-123, 2007.
- [12] J.A. Jones and M.J. Harrold. Test-Suite Reduction and Prioritization for Modified Condition/Decision Coverage. *IEEE Transactions on Software Engineering*, 29(3), pp. 195-209, 2003.
- [13] M. Martina and A. Bertolino. Using Spanning Sets for Coverage Testing. *IEEE Transactions on Software Engineering*, 29(11), pp.974-984, 2003.
- [14] S. McMaster and A. Memon. Call-Stack Coverage for GUI Test Suite Reduction. *IEEE Transactions on Software Engineering*, 34(1), pp. 99-115, 2008.
- [15] A.S. Namin and D.H. Andrews. The Influence of Size and Coverage on Test Suite Effectiveness. In *Proceedings of the 10th International Symposium on Software Testing and Analysis*, pp. 57-67, 2009.
- [16] M. Nashat and E. Khalid. Simulated Annealing and Genetic Algorithms for Optimal Regression Testing. *Journal of Software Maintenance: Research and Practice*. 11(1), pp. 19-34, 1999.
- [17] G. Rothermel, M.J. Harrold, J. Ostrin and C. Hong. An Empirical Study of the Effects of Minimization on the Fault-detection Capabilities of Test Suites. In *Proceedings of the 14th International Conference on Software Maintenance*, pp. 34-43, 1998.
- [18] S. Sprenkle, S. Sampath, E. Gibson, L. Pollock and A. Souter. An Empirical Comparison of Test Suite Reduction Techniques for User-Session-Based Testing of Web Applications. In *Proceedings of the 21st IEEE International Conference on Software Maintenance*, pp. 587-596, 2005.
- [19] S. Tallam and N. Gupta. A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization. In *Proceedings of the 6th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering*, pp. 35-42, 2005.
- [20] B. Vaysburg, L.H. Tahat and B. Korel. Dependence Analysis in Reduction of Requirement based Test Suites. In *Proceedings of the 3rd International Symposium on Software Testing and Analysis*, pp. 107-111, 2002.
- [21] W.E. Wong, J. Horgan, S. London and A. Mathur. Effect of Test Set Minimization on Fault Detection Effectiveness. *Software Practice and Experience*. 28(4), pp. 347-369, 1998.
- [22] Y.B. Yu, J.A. Jones and M.J. Harrold. An Empirical Study of the Effects of Test-suite Reduction on Fault Localization. In *Proceedings of the 30th International Conference on Software Engineering*, pp. 201-210, 2008.
- [23] X. Zhang, Q. Gu, X. Chen, J.X. Qi and D.X. Chen. A Study of Relative Redundancy in Test-Suite Reduction While Retaining or Even Improving Fault-Localization. In *Proceedings of the 25th Symposium On Applied Computing*, pp. 2229-2236, 2010.